

Java 8 Features - Cheat Sheet

Java 8 (released March 2014) was the biggest language change since generics. It brought lambdas, streams, a new date API, and a working `Optional` type. If you're writing modern Java, you're using Java 8 features whether you notice or not.

1. Lambda Expressions

Anonymous function syntax. Replaces the verbose anonymous inner class pattern for single-method interfaces.

Before Java 8:

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("running");  
    }  
};
```

With a lambda:

```
Runnable r = () -> System.out.println("running");
```

Common forms:

```
() -> doSomething()           // no args  
x -> x * 2                    // one arg, no parens needed  
(x, y) -> x + y               // multiple args  
(int x, int y) -> x + y       // explicit types  
(x) -> {                       // multi-line body  
    int z = x * 2;  
    return z + 1;  
}
```

The compiler infers types from the target functional interface.

2. Functional Interfaces

An interface with exactly one abstract method. That's the contract a lambda binds to.

The common ones live in `java.util.function`:

Interface	Signature	Use case
Function<T, R>	R apply(T t)	Transform input to output
Predicate<T>	boolean test(T t)	Check a condition
Consumer<T>	void accept(T t)	Use a value, no return
Supplier<T>	T get()	Produce a value
BiFunction<T, U, R>	R apply(T t, U u)	Two inputs, one output

Example:

```
Predicate<String> isShort = s -> s.length() < 5;
Function<String, Integer> length = String::length;
Consumer<String> printer = System.out::println;

isShort.test("hi");           // true
length.apply("hello");        // 5
printer.accept("done");       // prints "done"
```

For your own functional interfaces, use the `@FunctionalInterface` annotation. The compiler then enforces the single-abstract-method rule.

```
@FunctionalInterface
public interface Validator<T> {
    boolean isValid(T input);
}
```

3. Method References

Shorthand for a lambda that just calls one existing method.

Four forms:

```
String::length           // unbound instance method
System.out::println      // bound instance method
String::valueOf          // static method
ArrayList::new           // constructor
```

Example:

```
List<String> names = List.of("alice", "bob");

// Lambda
names.forEach(n -> System.out.println(n));

// Method reference
names.forEach(System.out::println);
```

Use when the lambda body is just one method call. Otherwise stick with the lambda.

4. Stream API

Pipeline-style processing for collections. Filter, map, reduce, collect.

```
List<String> names = List.of("alice", "bob", "charlie", "dave");

List<String> result = names.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());

// [ALICE, CHARLIE]
```

Common operations:

Operation	Purpose
<code>filter(Predicate)</code>	Keep only matching elements
<code>map(Function)</code>	Transform each element
<code>flatMap(Function)</code>	Flatten nested streams
<code>sorted()</code> / <code>sorted(Comparator)</code>	Sort the stream
<code>distinct()</code>	Remove duplicates
<code>limit(n)</code> / <code>skip(n)</code>	Take first n / drop first n
<code>forEach(Consumer)</code>	Run a side effect on each element (terminal)
<code>collect(Collector)</code>	Gather into a collection (terminal)
<code>reduce(...)</code>	Fold into a single value (terminal)
<code>count()</code> / <code>sum()</code> / <code>min()</code> / <code>max()</code>	Aggregate (terminal)

Streams are lazy. Nothing runs until a terminal operation fires.

Parallel streams:

```
list.parallelStream()  
    .filter(...)  
    .collect(...);
```

Use parallel streams only when the work per element is heavy enough to outweigh thread overhead. For small lists or quick operations, sequential is faster.

5. Default and Static Methods in Interfaces

Interfaces can now ship method bodies.

```
public interface Greeter {  
    String name();  
  
    default String greet() {  
        return "hello, " + name();  
    }  
  
    static Greeter named(String n) {  
        return () -> n;  
    }  
}
```

Why it matters: adding a method to an interface no longer breaks every implementer, as long as you can give it a sensible default. `Collection.stream()`, `Iterable.forEach()`, and `Map.getOrDefault()` were all added this way without breaking existing code.

Diamond conflict: if two interfaces define the same `default` method, the implementing class must override it to break the tie.

6. Optional

A container for “value or no value”. Safer than returning `null` and forces the caller to think about the empty case.

```
Optional<User> user = userRepo.findById(42);

user.ifPresent(u -> System.out.println(u.getName()));

String name = user
    .map(User::getName)
    .orElse("unknown");
```

Common methods:

- `of(value)` : wrap a non-null value, throws if null.
- `ofNullable(value)` : wrap a value that might be null.
- `empty()` : empty optional.
- `isPresent()` / `isEmpty()` : check the state.
- `get()` : extract the value, throws if empty. Avoid in favor of safer methods.
- `orElse(default)` : extract or return a default.
- `orElseGet(Supplier)` : same as `orElse` , but the default is lazy.
- `orElseThrow()` / `orElseThrow(Supplier)` : extract or throw.
- `map(Function)` / `flatMap(Function)` : transform if present.

Best used as a return type for methods that may not find a value. Don't use it as a field type or a method parameter.

7. New Date/Time API (java.time)

A clean replacement for the broken `Date` , `Calendar` , and `SimpleDateFormat` classes. Immutable and thread-safe. Designed around ISO-8601.

Key classes:

Class	What it represents
<code>LocalDate</code>	Date with no time or zone (2025-05-23)
<code>LocalTime</code>	Time with no date or zone (14:30:00)
<code>LocalDateTime</code>	Date + time, no zone
<code>ZonedDateTime</code>	Date + time + zone
<code>Instant</code>	Machine timestamp (epoch nanos)
<code>Duration</code>	Time-based amount (seconds, nanos)
<code>Period</code>	Date-based amount (years, months, days)
<code>DateTimeFormatter</code>	Parsing and formatting

Examples:

```


    LocalDate today = LocalDate.now();
    LocalDate tomorrow = today.plusDays(1);

    LocalDateTime meeting = LocalDateTime.of(2025, 6, 1, 14, 30);
    ZonedDateTime nyTime = meeting.atZone(ZoneId.of("America/New_York"));

    Duration elapsed = Duration.between(start, end);

    String formatted = today.format(DateTimeFormatter.ISO_LOCAL_DATE);
    LocalDate parsed = LocalDate.parse("2025-05-23");


```

All types are immutable. Methods like `plusDays()` return a new instance instead of mutating the original.

8. forEach on Collections

`Iterable.forEach(Consumer)` and `Map.forEach(BiConsumer)` are default methods that ship built-in iteration.

```


    List<String> list = List.of("a", "b", "c");
    list.forEach(System.out::println);

    Map<String, Integer> map = Map.of("x", 1, "y", 2);
    map.forEach((k, v) -> System.out.println(k + " = " + v));


```

Cleaner than the classic for-loop when you're just acting on each element.

Common Gotchas

- A lambda can only reference effectively-final local variables. Reassigning a variable used in a lambda breaks compilation.
 - Streams can only be consumed once. After a terminal operation runs, the stream is closed.
 - `Optional.get()` without checking `isPresent()` throws. Use `orElse` or `orElseThrow` instead.
 - `parallelStream()` shares the common `ForkJoinPool`. A blocking task in one parallel stream slows every other parallel stream in the JVM.
 - `LocalDateTime` has no timezone. If you need zones, use `ZonedDateTime` or `OffsetDateTime`.
 - Default methods in interfaces can't override methods from `Object` like `equals`, `hashCode`, or `toString`.
-

Quick Reference

Feature	One-line value
Lambdas	Short anonymous functions
Functional interfaces	The target type for lambdas
Method references	Lambdas that just call one method
Streams	Pipeline operations on collections
Default methods	Add to interfaces without breaking callers
Optional	Explicit “value or nothing” return type
<code>java.time</code>	The Date/Calendar replacement
<code>forEach</code>	One-liner iteration